Relatório da Base Aurora – MGPEB

Módulo de Gerenciamento de Pouso e Estabilização de Base (Aurora Siger)

Autor: Gabriel Bela dos Santos

Data: 14/10/2026

RM: 575188

1. Introdução

Este relatório documenta o protótipo do Módulo de Gerenciamento de Pouso e Estabilização de Base (MGPEB) para a missão Aurora Siger. O objetivo é simular a organização do pouso de módulos essenciais para uma colônia em Marte, aplicando lógica booleana, estruturas lineares (fila, listas e pilha), algoritmos clássicos de busca e ordenação, e uma modelagem matemática associada ao pouso. O protótipo também registra decisões em log (auditoria) e incorpora diretrizes de governança com perspectiva ESG.

Escopo funcional do MGPEB no protótipo: (i) cadastrar/receber atributos dos módulos por entrada interativa, (ii) ordenar e decidir pousos com base em regras booleanas, (iii) priorizar o módulo de energia em condições críticas de energia da base, (iv) estimar tempo de descida com gravidade marciana, e (v) permitir reprocessamento limitado de módulos em espera ou alerta quando as condições mudam.

2. Cenário de pouso e módulos definidos

O cenário considera cinco módulos pré-definidos: HAB-01 (Habitação), ENE-01 (Energia), LAB-01 (Laboratório Científico), LOG-01 (Logística) e MED-01 (Suporte Médico). Os módulos são inseridos na fila principal de autorização de pouso (queue). A cada ciclo, o MGPEB avalia condições operacionais e toma uma decisão: POUSADO, ESPERA ou ALERTA.

2.1 Atributos operacionais utilizados

Para cada módulo, o protótipo coleta por input: combustível (%), massa (kg), criticidade (1 a 5, sendo 1 mais crítico), ETA na órbita (min) e energia do módulo (kWh). O campo status indica o estado do módulo (AGUARDANDO, POUSADO, ESPERA ou ALERTA).

2.2 Estruturas de dados (lineares)

O MGPEB utiliza estruturas lineares coerentes com o conteúdo da disciplina: (a) uma fila principal (lista operando como queue) para módulos aguardando decisão; (b) listas auxiliares para módulos pousados, em espera ou em alerta; e (c) uma pilha de auditoria (stack) para registrar eventos e decisões em ordem LIFO (último evento no topo).

3. Regras de decisão com portas lógicas

As regras de autorização são expressas como funções booleanas usando AND, OR e NOT. As variáveis atmosféricas, disponibilidade da área e integridade dos sensores são informadas pelo usuário a cada ciclo (1=Sim, 0=Não). O combustível é comparado a um limiar mínimo.

3.1 Variáveis booleanas

combustivel_ok = (combustível ≥ LIMIAR_COMBUSTIVEL). atmosfera_ok, area_ok e sensores_ok são condições operacionais do ciclo. No protótipo, LIMIAR_COMBUSTIVEL = 20%.

3.2 Expressões booleanas

AUTORIZAR = combustivel_ok AND atmosfera_ok AND area_ok AND sensores_ok.

ALERTA = (NOT combustivel_ok) OR (NOT sensores_ok).

ESPERA = (NOT atmosfera_ok) OR (NOT area_ok).

3.3 Diagramas de portas lógicas

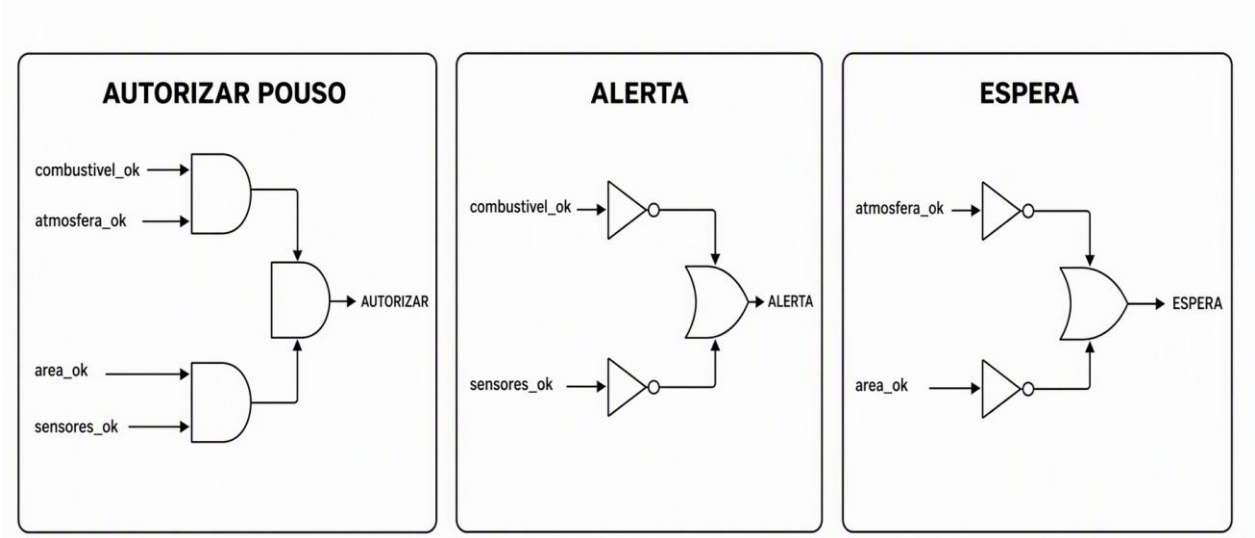


Diagrama lógico (texto):

AUTORIZAR: combustivel_ok AND atmosfera_ok AND area_ok AND sensores_ok

combustivel_ok
AND
atmosfera_ok
AND ----> AUTORIZAR
area_ok
AND
sensores_ok

ALERTA: (NOT combustivel_ok) OR (NOT sensores_ok)

NOT combustivel_ok
OR ----> ALERTA
NOT sensores_ok

ESPERA: (NOT atmosfera_ok) OR (NOT area_ok)

NOT atmosfera_ok
OR ----> ESPERA
NOT area_ok

PÚBLICA

4. Protótipo do MGPEB em Python

O protótipo foi implementado com estruturas condicionais IF/ELIF/ELSE refletindo diretamente as expressões booleanas. Antes de cada decisão, a fila principal é reorganizada por ordenação clássica (bubble sort). Em seguida, o primeiro módulo da fila é avaliado e classificado para pouso, espera ou alerta. Ao final, módulos em espera/alerta podem ser reprocessados com limite de tentativas igual ao total inicial de pendências (ESPERA + ALERTA), evitando loops.

4.1 Ordenação da fila (bubble sort)

A ordenação aplica os seguintes critérios: (1) criticidade (1 primeiro), (2) menor combustível primeiro em empate de criticidade, e (3) menor ETA em empate adicional. O bubble sort foi escolhido por ser um algoritmo clássico, simples e adequado para um conjunto pequeno de elementos (cinco módulos), facilitando entendimento e validação.

4.2 Regra didática de energia da base

O protótipo inclui uma regra adicional: se a energia atual da base estiver abaixo de um limiar crítico (LIMIAR_ENERGIA_BASE = 50,0 kWh), o módulo de energia ENE-01 é movido para o início da fila (prioridade operacional). Esta regra simula a governança técnica de continuidade operacional: em energia crítica, prioriza-se a infraestrutura que estabiliza a base.

Além disso, a energia da base é atualizada apenas quando o ENE-01 pousa (modelo mais realista do que somar energia de todos os módulos).

5. Modelagem matemática aplicada: Altura em função do tempo

Para apoiar a decisão de engenharia e o planejamento do pouso, foi modelada a altura do módulo durante a descida usando uma função quadrática (movimento com aceleração constante). O modelo de descida utiliza a função quadrática $h(t) = h0 + v0 \cdot t - (g/2) \cdot t^2$ com g fixo de Marte (3,71 m/s^2). O protótipo apresenta somente o tempo estimado até o pouso resolvendo $h(t)=0$, com a raiz positiva $t = (v0 + \sqrt{(v0^2 + 2 \cdot g \cdot h0)}) / g$.

PÚBLICA

5.1 Modelo

$$h(t) = h0 + v0 \cdot t - (g/2) \cdot t^2$$

Onde:
-h(t): altura (m)
-h0: altura inicial (m)
-v0: velocidade vertical inicial (m/s)
-g: gravidade (m/s^2), adotada como 3,71 m/s^2
-t: tempo (s) (Equação base (texto))

5.2 Tempo estimado até o pouso

O protótipo resolve o instante em que h(t) = 0 e apresenta apenas o tempo estimado até o pouso. Ao reorganizar a equação, obtém-se uma forma equivalente cuja raiz positiva pode ser escrita como:

$$t = (v0 + \sqrt{(v0^2 + 2 \cdot g \cdot h0)}) / g$$ (raiz positiva) (Tempo estimado (texto))

5.3 Análise qualitativa do comportamento

O termo $-(g/2) \cdot t^2$ torna a função uma parábola côncava para baixo. À medida que t aumenta, a altura decresce de forma acelerada. Aumentar h0 tende a aumentar o tempo até o pouso. Aumentar g (hipoteticamente) reduziria o tempo. Esse modelo auxilia na escolha do momento de acionamento de procedimentos de descida e na governança do fluxo de pousos (evitar sobrecarga simultânea de operações).

6. Contextualização histórica e arquitetural

A evolução da computação, desde os primeiros computadores de propósito geral até sistemas embarcados modernos, viabilizou o uso de controle computacional em operações críticas. Sistemas espaciais exigem previsibilidade, rastreabilidade, robustez e eficiência de recursos. Em uma missão em Marte, limitações de processamento, energia e tolerância a falhas influenciam escolhas de algoritmos e estruturas de dados.

No MGPEB, essas premissas se refletem em: (i) estruturas simples (fila/listas/pilha), (ii) algoritmos clássicos e facilmente auditáveis (bubble sort e varreduras lineares), (iii) regras booleanas claras e explicáveis, e (iv) logs de auditoria para rastreabilidade e governança. Mesmo

sendo um protótipo didático, o desenho favorece testabilidade e explicabilidade, importantes em sistemas de alta confiabilidade.

7. ESG e governança na Base Aurora Siger

O protótipo incorpora, de forma didática, princípios ESG por meio de critérios transparentes, rastreáveis e orientados à segurança:

7.1 Ambiental (E)

Critérios de pouso devem considerar minimização de poeira e impactos, evitando regiões de alto valor científico e reduzindo riscos de contaminação do ambiente marciano. A governança de janelas de pouso pode reduzir manobras desnecessárias e consumo de combustível.

7.2 Social (S)

A segurança das pessoas e a continuidade de suporte básico (habitação, energia e saúde) orientam priorizações. O uso de regras explicáveis e logs promove transparência e entendimento coletivo do porquê decisões foram tomadas, reduzindo assimetria de informação.

7.3 Governança (G)

A governança é reforçada por: regras booleanas auditáveis; registro de eventos em pilha de logs; reprocessamento controlado de pendências (limite de tentativas); e uma regra de prioridade energética que assegura continuidade operacional em condição crítica. Em um ambiente de fronteira, esses mecanismos ajudam a garantir decisões éticas, responsáveis e verificáveis.

8. Conclusão

O MGPEB prototipado demonstra, de forma integrada, como estruturas de dados lineares, lógica booleana, algoritmos clássicos e modelagem matemática podem apoiar a governança de pousos e a estabilização de uma base em Marte. O desenho prioriza clareza, auditabilidade e controle de riscos, com um mecanismo adicional de priorização do módulo de energia sob condição crítica. O protótipo é adequado para fins didáticos e pode ser estendido com telemetria real, critérios probabilísticos e controles redundantes.

ANEXOS

ANEXO A — Estruturas de dados utilizadas (listas, filas e pilhas)

Este anexo descreve como estruturas de dados lineares foram aplicadas no protótipo MGPEB, com exemplos concretos alinhados ao código.

A.1 Listas (list) — estados do sistema

Listas armazenam coleções de módulos por estado: lista_pousados, lista_espera e lista_alerta. Cada item é um dicionário contendo atributos do módulo. Exemplo: quando um módulo tem pouso autorizado, ele é adicionado com lista_pousados.append(modulo).

A.2 Fila (queue) — módulos aguardando autorização

A fila principal (fila_principal) é implementada como lista operando em modo FIFO: enfileirar usa append e desenfileirar remove o primeiro item (índice 0). Isso representa módulos aguardando decisão de pouso em sequência controlada.

Exemplo:
-enfileirar(fila_principal, modulo) # adiciona ao final
-atual = desenfileirar(fila_principal) # remove do início e processa (Exemplo concreto)

A.3 Pilha (stack) — log de auditoria

A pilha de logs (pilha_log) registra decisões e eventos para rastreabilidade. Cada evento é adicionado ao topo com log_evento(mensagem). Na finalização, imprime-se do último para o primeiro, permitindo auditoria das decisões mais recentes.

A.4 Reprocessamento de pendências (uso combinado)

O reprocessamento final combina listas e fila: módulos em ESPERA/ALERTA são retirados de suas listas, reinseridos na fila, reordenados e reavaliados com novas condições. Para

PÚBLICA

evitar loops infinitos, o limite de tentativas é definido como o total inicial de pendências (ESPERA + ALERTA) no início do reprocessamento.

ANEXO B — Link do repertório no GITHUB

https://github.com/Gabriel-Bela/-M-dulo-de-Gerenciamento-de-Pouso-e-Estabiliza-o-de-Base-MGPEB-.git

ANEXO A — Código em Python:

"""

MGPEB – Protótipo (Aurora Siger)

Inclui:

- Módulos fixos (HAB, ENE, LAB, LOG, MED) com preenchimento por input() (um por vez)

- Regras booleanas (AND/OR/NOT) para autorizar/alertar/esperar

- Fila/listas/pilha + bubble sort (clássico)

- Regra didática: energia da base crítica -> ENE-01 fura fila

- Altura x Tempo: g Marte fixo -> mostra só tempo estimado de pouso

- ESG

- Para testar o pouso de todos os módulos: 5 ciclos principais (fixo)

```
- Reprocessamento final de pendências (ESPERA/ALERTA) com limite de tentativas:
  max_tentativas = (qtd_espera_inicial + qtd_alerta_inicial)
- Finalização: lista POUSADOS / ESPERA / ALERTA
- Energia da base: só muda quando ENE-01 pousa (mais realista)
"""

# ============================
# CONSTANTES DIDÁTICAS
# ============================
G_MARTE = 3.71
LIMIAR_COMBUSTIVEL = 20
LIMIAR_ENERGIA_BASE = 50.0

# ============================
# 1) ESTRUTURAS + MÓDULOS
# ============================

def criar_modulo(nome, tipo):
    return {
        "nome": nome,
```

```
        "tipo": tipo,
        "combustivel": None,
        "massa": None,
        "criticidade": None,
        "eta_orbita_min": None,
        "energia_kwh": None,
        "status": "AGUARDANDO"
    }

fila_principal = []
lista_pousados = []
lista_espera = []
lista_alerta = []
pilha_log = []

def enfileirar(fila, modulo):
    fila.append(modulo)

def desenfileirar(fila):
    if len(fila) == 0:
```

```
        return None
    primeiro = fila[0]
    del fila[0]
    return primeiro


def log_evento(msg):
    pilha_log.append(msg)


MODULOS_FIXOS = [
    criar_modulo("HAB-01", "Habitação"),
    criar_modulo("ENE-01", "Energia"),
    criar_modulo("LAB-01", "Laboratório Científico"),
    criar_modulo("LOG-01", "Logística"),
    criar_modulo("MED-01", "Suporte Médico"),
]


def texto_esg():
    return (
        "Módulo de Gerenciamento de Pouso e Estabilização da Base Aurora Siger:\n"
        "Seção: Princípios ESG na Base Aurora\n"
```

```
        "Ambiental (E): reduzir poeira/impactos e evitar zonas de alto valor científico.\n"
        "Social (S): segurança e transparência nas decisões (logs e critérios claros).\n"
        "Governança (G): regras auditáveis, revisáveis e decisões justificáveis.\n"
        "Nessa missão temos como objetivo pousar todos os módulos com segurança de acordo com as variáveis e suas prioridades!\n"
    )

# ========================
# INPUTS
# ========================

def ler_int(msg, minimo=None, maximo=None):
    while True:
        txt = input(msg).strip()
        if txt.isdigit():
            v = int(txt)
            if minimo is not None and v < minimo:
                print(f"Valor inválido. Mínimo: {minimo}.")
                continue
            if maximo is not None and v > maximo:
```

```
            print(f"Valor inválido. Máximo: {maximo}.")
            continue
        return v
    print("Entrada inválida. Digite um número inteiro.")


def ler_float(msg, minimo=None, maximo=None):
    while True:
        txt = input(msg).strip().replace(",", ".")
        try:
            v = float(txt)
            if minimo is not None and v < minimo:
                print(f"Valor inválido. Mínimo: {minimo}.")
                continue
            if maximo is not None and v > maximo:
                print(f"Valor inválido. Máximo: {maximo}.")
                continue
            return v
        except ValueError:
            print("Entrada inválida. Digite um número (ex.: 12.5).")
```

```
def preencher_dados_modulos(modulos):
    print("\n=== PREENCHIMENTO DOS MÓDULOS (um por vez) ===")
    for m in modulos:
        print(f"\n--- {m['nome']} | {m['tipo']} ---")
        m["combustivel"] = ler_float("Combustível (% 0 a 100): ", 0, 100)
        m["massa"] = ler_float("Massa (kg): ", 1, 999999999)
        m["criticidade"] = ler_int("Criticidade (1=mais crítico ... 5=menos crítico): ", 1, 5)
        m["eta_orbita_min"] = ler_int("ETA na órbita (min): ", 0, 999999999)
        m["energia_kwh"] = ler_float("Energia do módulo (kWh): ", 0, 999999999)

    log_evento("Dados preenchidos para todos os módulos pré-definidos.")

# ==========================
# 2) PORTAS LÓGICAS (BOOLEANAS)
# ==========================

def avaliar_regras(modulo, atmosfera_ok, area_ok, sensores_ok):
    combustivel_ok = (modulo["combustivel"] >= LIMIAR_COMBUSTIVEL)
    autorizar = combustivel_ok and atmosfera_ok and area_ok and sensores_ok
    alerta = (not combustivel_ok) or (not sensores_ok)
```

```
    espera = (not atmosfera_ok) or (not area_ok)
    return autorizar, alerta, espera

def ler_condicoes_operacionais():
    print("\nCondições operacionais (1=Sim / 0=Não):")
    atmosfera_ok = (ler_int("Atmosfera OK? ", 0, 1) == 1)
    area_ok = (ler_int("Área de pouso disponível? ", 0, 1) == 1)
    sensores_ok = (ler_int("Sensores OK? ", 0, 1) == 1)
    return atmosfera_ok, area_ok, sensores_ok

# ==========================
# 3) ORDENAÇÃO (BUBBLE SORT) + REGRA ENERGIA
# ==========================

def bubble_sort_por_criticidade_e_combustivel(fila):
    ordenada = []
    for item in fila:
        ordenada.append(item)

    n = len(ordenada)
```

```
for i in range(n):
    trocou = False
    for j in range(0, n - 1 - i):
        a = ordenada[j]
        b = ordenada[j + 1]

        cond1 = (a["criticidade"] > b["criticidade"])
        cond2 = (a["criticidade"] == b["criticidade"] and a["combustivel"] > b["combustivel"])
        cond3 = (
            a["criticidade"] == b["criticidade"]
            and a["combustivel"] == b["combustivel"]
            and a["eta_orbita_min"] > b["eta_orbita_min"]
        )

        if cond1 or cond2 or cond3:
            ordenada[j], ordenada[j + 1] = ordenada[j + 1], ordenada[j]
            trocou = True

    if not trocou:
        break
```

```
    return ordenada


def aplicar_regra_energia_fura_fila(fila, energia_base_kwh):
    if energia_base_kwh >= LIMIAR_ENERGIA_BASE:
        return

    idx = -1
    for i in range(len(fila)):
        if fila[i]["nome"] == "ENE-01":
            idx = i
            break
    if idx == -1:
        return

    ene = fila[idx]
    del fila[idx]
    fila.insert(0, ene)
    log_evento(f"Regra energia crítica: ENE-01 priorizado (energia_base={energia_base_kwh}
kWh).")
```

```
def reordenar_fila(fila, energia_base_kwh):
    fila[:] = bubble_sort_por_criticidade_e_combustivel(fila)
    log_evento("Fila reordenada por criticidade e combustível (bubble sort).")
    aplicar_regra_energia_fura_fila(fila, energia_base_kwh)


# ==========================
# 4) ALTURA x TEMPO (tempo estimado)
# ==========================


def tempo_estimado_pouso(h0, v0, g):
    d = (v0 ** 2) + (2 * g * h0)
    if d < 0 or g <= 0:
        return None
    raiz = d ** 0.5
    t = (v0 + raiz) / g
    return t


def calcular_tempo_pouso_por_input():
    print("\n=== TEMPO ESTIMADO PARA POUSO (Altura x Tempo) ===")
```

```
    print(f"Gravidade de Marte fixa: g = {G_MARTE} m/s²")
    h0 = ler_float("Altura inicial h0 (m): ", 0.01, 10**9)
    v0 = ler_float("Velocidade vertical inicial v0 (m/s) (use 0 se quiser): ", -10**6, 10**6)

    t = tempo_estimado_pouso(h0, v0, G_MARTE)
    if t is None:
        print("Não foi possível calcular o tempo (valores inválidos).")
        log_evento("Falha no cálculo do tempo de pouso.")
        return None

    print(f"Tempo estimado até o pouso: {t:.2f} s")
    log_evento(f"Tempo de pouso estimado: {t:.2f} s.")
    return t

# ==========================
# DECISÃO DE POUSO (reutilizável)
# ==========================

def decidir_e_alocar(modulo, atmosfera_ok, area_ok, sensores_ok, energia_base_kwh):
    """
```

```
Aplica as regras booleanas e move o módulo para a lista correta.

Retorna energia_base_kwh (pode mudar se ENE-01 pousar).

"""

autorizar, alerta, espera = avaliar_regras(modulo, atmosfera_ok, area_ok, sensores_ok)


if autorizar:

    modulo["status"] = "POUSADO"

    lista_pousados.append(modulo)

    print("DECISÃO: POUSO AUTORIZADO.")

    log_evento(f"POUSO AUTORIZADO: {modulo['nome']}.")


    # Energia da base só muda quando ENE-01 pousa

    if modulo["nome"] == "ENE-01":

        energia_base_kwh = energia_base_kwh + modulo["energia_kwh"]

        log_evento(f"Energia base após pouso do ENE-01: {energia_base_kwh} kWh.")

    else:

        log_evento("Energia base não alterada (módulo não é ENE-01).")


elif alerta:

    modulo["status"] = "ALERTA"
```

```
            lista_alerta.append(modulo)
            print("DECISÃO: ALERTA.")
            log_evento(f"ALERTA: {modulo['nome']}.")

        elif espera:
            modulo["status"] = "ESPERA"
            lista_espera.append(modulo)
            print("DECISÃO: ESPERA.")
            log_evento(f"ESPERA: {modulo['nome']}.")

        else:
            modulo["status"] = "BLOQUEADO"
            lista_alerta.append(modulo)
            print("DECISÃO: BLOQUEADO (caso residual).")
            log_evento(f"BLOQUEADO: {modulo['nome']}.")

        return energia_base_kwh

    # ========================
    # CICLOS PRINCIPAIS
```

```
# ==========================


def simular_ciclo(energia_base_kwh):
    if len(fila_principal) == 0:
        print("\nFila vazia. Nada para processar.")
        log_evento("Ciclo com fila vazia.")
        return energia_base_kwh

    reordenar_fila(fila_principal, energia_base_kwh)
    atual = desenfileirar(fila_principal)

    print(
        f"\nProcessando: {atual['nome']} | {atual['tipo']} | "
        f"crit={atual['criticidade']} | comb={atual['combustivel']}% |
energia_mod={atual['energia_kwh']} kWh"
    )
    print(f"Energia base: {energia_base_kwh} kWh (crítico se < {LIMIAR_ENERGIA_BASE}
kWh)")

    atmosfera_ok, area_ok, sensores_ok = ler_condicoes_operacionais()
    return decidir_e_alocar(atual, atmosfera_ok, area_ok, sensores_ok, energia_base_kwh)
```

```
# ========================
# REPROCESSAMENTO FINAL (ESPERA/ALERTA) com limite de tentativas
# ========================

def reprocessar_pendencias(energia_base_kwh):
    max_tentativas = len(lista_espera) + len(lista_alerta)
    tentativas_feitas = 0

    if max_tentativas == 0:
        return energia_base_kwh

    log_evento(f"Reprocessamento iniciado: limite de tentativas = {max_tentativas}.")

    while (len(lista_espera) > 0 or len(lista_alerta) > 0) and (tentativas_feitas < max_tentativas):
        print("\n=== PENDÊNCIAS DETECTADAS ===")
        print(f"Em ESPERA: {len(lista_espera)} | Em ALERTA: {len(lista_alerta)}")
        print(f"Tentativas restantes: {max_tentativas - tentativas_feitas}")

        opc = ler_int("Tentar reavaliar pouso dessas pendências? (1=Sim / 0=Não): ", 0, 1)
```

```
if opc == 0:
    log_evento("Usuário optou por não reprocessar pendências.")
    break

escolha = ler_int("Reprocessar qual primeiro? (1=ESPERA / 2=ALERTA): ", 1, 2)

if escolha == 1 and len(lista_espera) == 0:
    print("Não há módulos em ESPERA. Indo para ALERTA.")
    escolha = 2
if escolha == 2 and len(lista_alerta) == 0:
    print("Não há módulos em ALERTA. Indo para ESPERA.")
    escolha = 1

if escolha == 1:
    modulo = lista_espera[0]
    del lista_espera[0]
    origem = "ESPERA"
else:
    modulo = lista_alerta[0]
    del lista_alerta[0]
```

```
            origem = "ALERTA"

        print(f"\nReavaliando (antes em {origem}): {modulo['nome']} | {modulo['tipo']}")

        modulo["status"] = "AGUARDANDO"
        enfileirar(fila_principal, modulo)
        log_evento(f"Pendência reinserida na fila: {modulo['nome']} (origem={origem}).")

        reordenar_fila(fila_principal, energia_base_kwh)
        atual = desenfileirar(fila_principal)

        print(f"\nProcessando novamente: {atual['nome']} | comb={atual['combustivel']}% |
crit={atual['criticidade']}")
        print(f"Energia base: {energia_base_kwh} kWh (crítico se < {LIMIAR_ENERGIA_BASE}
kWh)")

        atmosfera_ok, area_ok, sensores_ok = ler_condicoes_operacionais()
        energia_base_kwh = decidir_e_alocar(atual, atmosfera_ok, area_ok, sensores_ok,
energia_base_kwh)

        tentativas_feitas += 1
```

```
        log_evento(f"Tentativa de reprocessamento {tentativas_feitas}/{max_tentativas}
concluída.")


    if tentativas_feitas >= max_tentativas and (len(lista_espera) > 0 or len(lista_alerta) > 0):
        print("\nLimite de tentativas atingido. Permanecem pendências sem reavaliação.")
        log_evento("Limite de tentativas atingido no reprocessamento. Pendências remanescentes.")


    return energia_base_kwh


# ==========================
# RELATÓRIOS
# ==========================


def imprimir_estado():
    print("\n=== ESTADO ATUAL ===")
    print("Fila (aguardando):")
    for m in fila_principal:
        print(
            f" - {m['nome']} | {m['tipo']} | crit={m['criticidade']} | comb={m['combustivel']}% | "
            f"ETA={m['eta_orbita_min']} | energia_mod={m['energia_kwh']} kWh"
```

```
        )

    print(f"\nPousados: {len(lista_pousados)} | Em espera: {len(lista_espera)} | Em alerta:
{len(lista_alerta)}")


def listar_resultados_finais():
    print("\n=== RESULTADOS FINAIS (LISTAGEM COMPLETA) ===")


    print("\nPOUSADOS (ordem de pouso):")
    if len(lista_pousados) == 0:
        print(" - Nenhum módulo pousado.")
    else:
        for i in range(len(lista_pousados)):
            m = lista_pousados[i]
            print(f" {i+1}. {m['nome']} | {m['tipo']} | crit={m['criticidade']} |
comb={m['combustivel']}%")


    print("\nESPERA:")
    if len(lista_espera) == 0:
        print(" - Nenhum módulo em espera.")
    else:
        for m in lista_espera:
```

```
            print(f" - {m['nome']} | {m['tipo']} | crit={m['criticidade']} | comb={m['combustivel']}%")

    print("\nALERTA:")
    if len(lista_alerta) == 0:
        print(" - Nenhum módulo em alerta.")
    else:
        for m in lista_alerta:
            print(f" - {m['nome']} | {m['tipo']} | crit={m['criticidade']} | comb={m['combustivel']}%")

def imprimir_logs():
    print("\n=== LOG (PILHA) - mais recente primeiro ===")
    i = len(pilha_log) - 1
    while i >= 0:
        print(" -", pilha_log[i])
        i -= 1

# ==========================
# MAIN
# ==========================
```

```
def main():
    print(texto_esg())
    print("MGPEB - Protótipo com reprocessamento final (ESPERA/ALERTA)")
    print(f"Limiar combustível OK: {LIMIAR_COMBUSTIVEL}%")
    print(f"Limiar energia crítica base: {LIMIAR_ENERGIA_BASE} kWh\n")

    energia_base_kwh = ler_float("Energia atual inicial da base (kWh): ", 0, 999999999)
    log_evento(f"Energia inicial base: {energia_base_kwh} kWh.")

    preencher_dados_modulos(MODULOS_FIXOS)
    for m in MODULOS_FIXOS:
        enfileirar(fila_principal, m)
    log_evento("Módulos enfileirados na fila principal.")

    calcular_tempo_pouso_por_input()

    # Para testar o pouso de todos os módulos: 5 ciclos (fixo)
    ciclos = 5
    for c in range(1, ciclos + 1):
        print(f"\n--- CICLO PRINCIPAL {c} ---")
```

```
        energia_base_kwh = simular_ciclo(energia_base_kwh)
        imprimir_estado()

    # Reprocessamento de pendências
    energia_base_kwh = reprocessar_pendencias(energia_base_kwh)

    print("\n=== FINALIZAÇÃO ===")
    imprimir_estado()
    listar_resultados_finais()
    imprimir_logs()

main()
```

REFERÊNCIAS

CORMEN, T. H. et al. Algoritmos: Teoria e Prática. (Conceitos de ordenação e busca).

Documentação Python. Estruturas de dados (listas) e controle de fluxo (if/elif/else).